

TP12 : programmation dynamique

ZFAI

1 Introduction

Dans de nombreux problèmes informatiques, une implémentation maladroite entraîne une redondance des calculs. Pour éviter ce type de problème, il est possible d'utiliser une manière de programmer, la programmation dynamique.

2 Exemples

2.1 Exemple : une programmation maladroite

On considère le programme suivant :

```
def suite(n) :  
    u = 5  
    for i in range(1,n+1) :  
        u = i*u + 2  
    return u  
  
def calcul_liste(n) :  
    L = []  
    for i in range(n+1) :  
        L.append(suite(i))  
    return L
```

Évaluons la complexité de ces différentes fonctions. Pour la fonction `suite`, le calcul de `suite(n)` on a une complexité en $O(n)$ opérations élémentaires. Il en résulte que le calcul de `calcul_liste(n)` se fait en $O(n^2)$ opérations élémentaires. Mais `calcul_liste` n'est pas optimale : lorsque l'on calcule `suite(i)`, tous les termes `suite(k)` pour $k < i$ sont recalculés. Il est donc possible d'améliorer la complexité en prenant en compte cette remarque. Ce qui donne le programme suivant :

```
def calcul_liste_opt(n) :  
    L = [5]  
    for i in range(1,n+1) :  
        u = i*L[i-1] + 2  
        L.append(u)  
    return L
```

Pour cette nouvelle version, on trouve comme complexité un $O(n)$ opérations élémentaires pour `calcul_liste_opt(n)`.

2.2 Exemple : la suite des nombres de Catalan

On considère la suite définie par :

$$C_0 = 1, \forall n \in \mathbb{N}, C_{n+1} = \sum_{k=0}^n C_k C_{n-k} \quad (1)$$

En calculant les premiers termes, on trouve : 1, 1, 2, 5, 14, 42...

Une écriture récursive naïve donne :

```
def catalan_naif(n) :  
    if n == 0 :  
        return 1  
    else :  
        S = 0  
        for i in range(n) :  
            S = S + catalan_naif(i) * catalan_naif(n-1-i)  
        return S
```

Il est clair que l'on a une explosion combinatoire dans ce contexte. Pour éviter ce type de problème, on peut procéder comme précédemment :

```
def catalan_iter(n) :
    C = [1]
    for k in range(1,n+1) :
        S = 0
        for i in range(k) :
            S = S + C[i] * C[k-1-i]
        C.append(S)
    return C[n]
```

Pour écrire récursivement ce problème, on peut par exemple utiliser un dictionnaire ou une liste en variable globale. Par exemple :

```
dico_cat = {0 : 1}
def catalan(n) :
    if n in dico_cat :
        return dico_cat[n]
    else :
        S = 0
        for i in range(n) :
            S = S + catalan[i] * catalan[n-1-i]
        dico_cat[n] = S
        # très important : stocke le résultat dans le dico
    return S
```

2.3 Exemple : une pyramide d'entiers

On considère une pyramide d'entiers naturels par exemple comme celle-ci :

```

      3
     4 2
    5 1 3
   6 4 5 2
  3 1 2 3 7
 8 3 4 1 3 2
```

Étant donné un chemin simple $C = c_1, \dots, c_n$ vérifiant :

- pour tout $i \in \{1, \dots, n\}$, c_i est une case de la i -ème ligne,
- pour tout $i \in \{1, \dots, n-1\}$, c_i et c_{i+1} sont des cases adjacentes,

on note $S(C)$ la somme des éléments apparaissant dans C . On cherche alors à maximiser la somme S .

Une première approche, est de considérer tous les chemins C et de calculer pour chaque chemin $S(C)$ et de garder le maximum. Mais si on procède ainsi, on obtient une complexité exponentielle. Le nombre de tel chemin est minoré par 2^n .

Mais cette façon de procéder n'optimise pas les calculs. Pour améliorer la complexité, on cherche une relation de récurrence que l'on peut exploiter.

Pour cela, on peut remarquer que si on note $S_{i,j}$ la somme maximale d'un chemin commençant par la première case et aboutissant à la j -ième case de la ligne i , on a alors :

$$S_{i,j} = \max(S_{i-1,j}, S_{i-1,j-1}, S_{i-1,j+1}) + C_{i,j}$$

avec $C_{i,j}$ qui est le contenu de la case (i,j) et comme convention $S_{i,j} = 0$ si celui-ci n'est pas bien défini par rapport aux indices. Le résultat recherché est alors le maximum des $S_{n,j}$.

Pour programmer de manière efficace, on exploite alors la récurrence. Ceci donne alors comme fonction :

```
def optimale(P) :
    N = len(P)
    L = [[P[i][j] for i in range(j+1)] for j in range(N)]
    for i in range(1,N) :
        for j in range(0,i+1) :
            M = [0]
            for k in range(j-1,j+2) :
                if (k >= 0) and (k < i) :
                    M.append(L[i-1][k])
            L[i][j] = max(M) + P[i][j]
    return max(L[N-1])
```

En terme de complexité, on obtient du $O(n^2)$ pour une pyramide ayant n lignes.

3 Travaux pratiques

Dans ce sujet, un **système monétaire** est un ensemble de n entiers naturels non nuls distincts deux à deux $D = \{d_0, d_1, \dots, d_{n-1}\}$ avec $n > 0$ et $d_{n-1} = 1$. Les d_i sont les **dénominations** du système. Par convention, les dénominations sont présentées par ordre **décroissant**. Par exemple, le système de l'euro est l'ensemble $\{500, 200, 100, 50, 20, 10, 5, 2, 1\}$.

Une **somme** (d'argent) est une suite finie $[e_0, e_1, \dots, e_{m-1}]$ d'entiers appartenant à D . Les éléments de cette suite sont des **espèces**. On remarquera que deux espèces d'une somme peuvent porter la même dénomination, qu'une somme peut être vide, et qu'il n'y a pas nécessairement d'espèces de dénomination 1 dans une somme. Par convention, les espèces sont présentées par ordre de dénominations décroissantes.

La **valeur** d'une somme S , notée $\text{val}(S)$ est la somme arithmétique de ses espèces, tandis que sa **taille**, notée $|S|$, est le nombre de ses éléments (l'entier m ci-dessus). Étant donné un entier naturel v , une somme de valeur v est un **représentant** de v . Par exemple, un portefeuille qui contient trois billets de 10 euros, deux billets de 5 euros et une pièce de 1 euro, représente une somme notée $[10, 10, 10, 5, 5, 1]$. Sa valeur est 41 et sa taille 6.

Une somme S est **extraite** d'une autre P dite portefeuille, si et seulement si S est une suite extraite de P . Par exemple, si on a un portefeuille $P = [10, 10, 10, 5, 5, 1]$, $S = [10, 5]$ est extraite de P .

Dans la suite, on représente en Python un système D par une liste d'entiers strictement décroissante terminant par 1, et une somme S par une liste décroissante d'éléments de D .

On supposera (pour les arguments) et on garantira (pour les résultats) les propriétés suivantes :

- tous les entiers présents dans les listes sont strictement positifs,
- toutes les listes sont triées en ordre décroissant,
- les listes représentant un système contiennent des entiers distincts deux à deux et leur dernier élément est 1.

Pour commencer

1. Écrire la fonction `valeur(S)` prenant en argument une somme et qui renvoie la valeur correspondante. On n'utilisera pas la fonction `sum` de Python.

Payer le compte exact

Dans cette partie on considère le problème du paiement exact. Dans les termes du préambule, cela revient, étant donné un entier naturel p , à trouver un représentant de p .

Une démarche possible pour payer le prix p est la démarche dite **gloutonne**, que l'on peut décrire informellement ainsi :

- donner l'espèce la plus élevée possible, c'est-à-dire la plus grande dénomination d disponible et vérifiant $d \leq p$;
- recommencer en enlevant l'espèce donnée au prix à payer, c'est-à-dire poser p égal à $p - d$.

Le processus s'arrête alors lorsque le prix initial est entièrement payé.

2. Dans cette question, on suppose que l'acheteur dispose toujours des espèces dont il a besoin.
 - (a) Montrer que la démarche gloutonne réussit toujours.
 - (b) Écrire une fonction `glouton(D, p)` prenant en arguments une liste D représentant un système et un entier positif p représentant un prix à payer et qui renvoie la liste représentant la somme obtenue à l'aide de la démarche gloutonne.
3. On tient cette fois compte des ressources de l'acheteur. Dans les termes du préambule cela revient à trouver une somme ayant pour valeur le prix à payer et extraite d'une somme donnée dite portefeuille.
 - (a) Montrer, à l'aide d'un exemple utilisant le système européen $([500, 200, 100, 50, 20, 10, 5, 2, 1])$, que la stratégie gloutonne peut échouer pour un prix donné, même s'il est possible de payer compte tenu des ressources disponibles.
 - (b) Écrire une fonction `paye_glouton(Pf, p)` prenant en arguments une liste Pf représentant une somme (un portefeuille), un entier positif p représentant un prix à payer et qui renvoie la somme extraite de Pf de valeur p obtenue à l'aide de la démarche gloutonne. Dans le cas où cette démarche échoue, la liste renvoyée est vide.
4. On cherche à compter le nombre de façons de payer un certain prix p étant donné un portefeuille Pf .
 - (a) Écrire une fonction `compte_paiements(Pf, p)` qui prend en arguments une liste Pf représentant un portefeuille et un entier naturel p représentant un prix et qui renvoie le nombre de façons de payer p à l'aide des espèces de Pf . Autrement dit cette fonction renvoie le cardinal de l'ensemble des représentants de p extraits de Pf . On ne cherchera pas à optimiser en termes de calculs dans cette première version. NOTE : Deux espèces de même dénomination sont distinguées. Ainsi, si le portefeuille est $[2, 2, 2]$ et le prix est 2, alors il y a trois façons de payer.

- (b) Pour optimiser la fonction précédente, une façon de procéder est d'utiliser un dictionnaire. Cependant, les listes n'étant pas hachables, on ne peut pas les utiliser comme clé. Pour cette question, on remplace la liste `Pf` par un tuple.

Écrire une fonction `compte_paiements_opt(Pf,p)` prenant en arguments un tuple `Pf` représentant un portefeuille et un entier positif `p` représentant un prix et qui renvoie le nombre de façons de payer `p` à l'aide de `Pf`. Dans cette version, on pourra s'aider d'un dictionnaire ou d'une matrice. Dans tous les cas, on optimisera le nombre de calculs.

Payer le compte exact et optimal

Une somme est *optimale* lorsque sa taille est minimale parmi un ensemble de sommes de valeur donnée. Par exemple, la somme $S = [20, 20, 1]$ montre que le portefeuille $P = [10, 10, 10, 5, 5, 1]$ n'est pas optimal parmi les sommes de valeur 41. En effet, la taille de S est 3 et elle est strictement inférieure à la taille de P .

5. On suppose qu'on dispose d'une infinité d'espèces pour chaque dénomination de D . On fixe p un entier strictement positif. L'objectif est alors de trouver la taille d'une somme optimale de p qu'on note $t(p, D)$.

- (a) Justifier brièvement qu'on a :

- $t(0, D) = 0$,
- $\forall p \in \mathbb{N}, t(p, [1]) = p$,
- $\forall p \in \mathbb{N}^*, t(p, D) = \min(1 + t(p - d_0, D), t(p, D \setminus \{d_0\}))$.

- (b) À l'aide de la programmation dynamique et de la récurrence précédente, écrire une fonction `taille_opt(D,p)` prenant en arguments une liste D représentant un système monétaire et un entier naturel p représentant un prix et qui renvoie la taille d'une somme optimale de p . On veillera à optimiser les calculs.

- (c) En adaptant la fonction précédente, écrire une fonction `somme_opt(D,p)` prenant en arguments une liste D représentant un système monétaire et un entier naturel strictement positif p représentant un prix et qui renvoie une somme optimale S pour le prix p .

6. Pour cette question, on se fixe un portefeuille `Pf`. Étant donné un prix p , trouver une somme optimale relative au portefeuille `Pf` consiste à trouver somme S extraite de `Pf` de valeur p , et de taille minimale, c'est-à-dire que toute somme extraite S' de `Pf` de valeur p a une taille plus grande que S .

En vous inspirant des questions précédentes et à l'aide de la programmation dynamique, écrire une fonction `optimale(Pf,p)` prenant en arguments une liste `Pf` représentant un portefeuille et un entier naturel p représentant un prix et qui renvoie une somme optimale relative au portefeuille `Pf`. Dans le cas où une telle somme n'existe pas, la liste renvoyée est vide.

4 Pour les plus rapides

La méthode par programmation dynamique est très générale. On donne quelques exercices que l'on peut résoudre par cette méthode.

Exercice 1. On considère la suites d'entiers $(u_k)_{k \in \mathbb{N}}$ définie par

$$u_0 = 115, \forall k \in \mathbb{N}, u_{k+1} = 15091 \cdot u_k \mod 64007.$$

Donner les valeurs de $u_{10}, u_{100}, u_{1000}$

Exercice 2. Étant donnée une suite finie de nombres $u_0, u_1, \dots, u_n$, une plus grande sous-suite croissante est une sous-suite croissante de $u_0, \dots, u_n$ de longueur maximale, c'est-à-dire toute sous-suite qui la contient strictement n'est pas croissante.

1. Expliciter une récurrence reliant une plus grande sous-suite croissante de $u_0, \dots, u_n$ avec une plus grande sous-suite croissante de $u_1, \dots, u_n$.
2. Écrire une fonction `plus_longue_ssc(u)` qui renvoie une plus grande sous-suite croissante de u .

Exercice 3. Étant données deux chaînes $a = a_0 \cdots a_{n-1}$ et $b = b_0 \cdots b_{m-1}$, une plus longue sous-suite commune entre a et b est l'une de ces trois formes :

- $c = a_0 c'$, dans le cas où $a_0 = b_0$ et c' est une plus longue sous-suite commune entre $a_1 \cdots a_{n-1}$ et $b_1 \cdots b_{m-1}$,
- c une est plus longue sous-suite commune entre a et $b_1 \cdots b_{m-1}$
- c une est plus longue sous-suite commune entre $a_1 \cdots a_{n-1}$ et b .

En exploitant cette remarque, déterminer une plus longue sous-suite commune à l'aide de la programmation dynamique.